Texturing the Triangle Sets

An implementation plan for adding image textures to the WebGL rasterizer

Summary

This report describes how to add texture mapping to the triangle sets in the rasterizer while keeping its current file structure: one file of configuration constants, one of shared global state, a model loader, a mesh generator, a shader-setup module, separate vertex and fragment shader sources, an input module, a renderer and an entry point. Ellipsoids stay untextured.

The scene data already provides what is needed. Each triangle set lists a texture coordinate (UV) for every vertex and names an image file in its material. The work is to carry those UVs to the GPU, load each image into a texture object, and sample it in the fragment shader, where the texel is combined with the existing Blinn-Phong lighting.

Recommended approach, in one sentence: add a third per-vertex attribute for UVs, load each image once into a cached texture (shown as a 1x1 placeholder until the image arrives), and **modulate** the lit color by the texel, controlled by a uniform flag so ellipsoids keep rendering exactly as they do now.

How texturing fits the pipeline

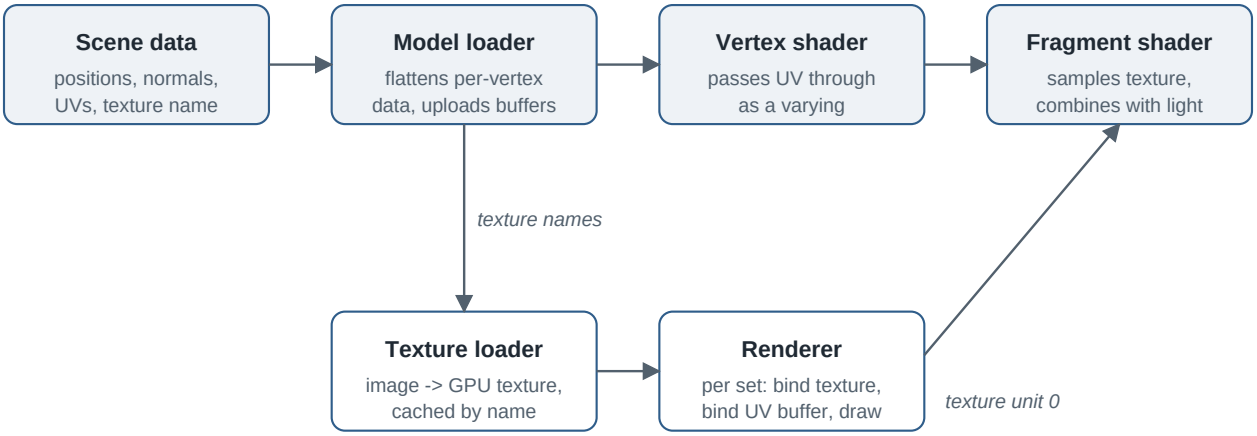

Figure 1. Data flow for textured triangle sets. The top row is the existing per-vertex path, extended with UVs. The bottom row is new.

A texture coordinate is a 2D position in the image, from (0, 0) at one corner to (1, 1) at the opposite corner. The vertex shader passes each vertex's UV through unchanged; the rasterizer interpolates it across the triangle with perspective correction, so every fragment receives its own UV. The fragment shader uses that UV to read (sample) a color from the texture bound to a texture unit, then combines it with lighting.

Changes by module

Each row below names a module by its role in the existing structure. The order is the order data flows, which is also a sensible implementation order.

Module	What to add
Configuration	The base location the texture images are fetched from (the same host as the scene data), and optionally the default combine mode.
Shared state	Per-set lists for UV buffers and texture objects; a texture cache keyed by image name; locations for the new UV attribute, the sampler uniform and the use-texture flag; a display toggle for texturing if you want a checkbox.
Model loader	Flatten each set's UVs into a float array alongside positions and normals and upload them into a new vertex buffer. Ask the texture loader for the set's texture by name.
Texture loader (new helper)	Create a texture, fill it with a 1x1 placeholder pixel, start the image download, and on load upload the image and set filtering and wrapping. Return cached textures for names already seen. This can live in the model loader or in a small new module.
Vertex shader	A new 2D input attribute for the UV and a 2D varying that copies it out.
Fragment shader	The matching varying, a 2D sampler uniform and a boolean use-texture uniform; sample the texture and combine it with the lighting terms.
Shader setup	Look up the UV attribute and the two new uniforms. Point the sampler at texture unit 0 once, since it never changes.
Renderer	For each triangle set: set the use-texture flag on, bind the set's texture to unit 0, bind its UV buffer and describe it to the UV attribute, then draw. For ellipsoids: flag off and no UV buffer (see 'The ellipsoid problem').
Input / page	Optional: a checkbox or key that switches texturing or the combine mode, wired the same way as the existing display toggles.

Step-by-step implementation

1. Carry UVs from the data to the GPU

In the loop that already flattens vertex positions and normals, flatten the UVs the same way: two floats per vertex, in the same vertex order, so the indices in the triangle list address position, normal and UV together. Upload the array into a new static array buffer per set and store it beside the existing vertex and normal buffers.

- Validate that the UV list has the same length as the vertex list. If a set has no UVs, either skip texturing for that set or fill with zeros, and log a warning.
- Keep UVs as given. Do not normalize or clamp them; values outside 0 to 1 are meaningful when the wrap mode is repeat.

2. Load images into texture objects

Image loading is asynchronous, but the loader and renderer are synchronous. The standard solution is a placeholder:

1. Create the texture and immediately upload a single opaque pixel (white works best with modulation, because it leaves the lit color unchanged). The set can render right away.
2. Create an image object, mark it as an anonymous cross-origin request, and set its source to the base location plus the material's texture name.
3. When the image finishes loading, bind the texture again, enable vertical flipping for the upload, upload the image, and set the sampling parameters (next step).
4. If the image fails to load, keep the placeholder and log the name, so the scene still renders.

Cache textures by image name. Several sets may share an image, and re-uploading the same image wastes memory and bandwidth.

3. Set filtering and wrapping

WebGL 1 treats textures whose width or height is not a power of two (NPOT) specially. An NPOT texture is incomplete, and samples as black, unless it uses clamp-to-edge wrapping in both directions and a minification filter without mipmaps. Decide per image after it loads:

Image size	Wrapping	Minification	Magnification
Both sides power of two	Repeat (or as needed)	Generate mipmaps; trilinear filtering	Linear
Any side not power of two	Clamp to edge, both axes	Linear, no mipmaps	Linear

If tiling NPOT images is a requirement, resize them to the next power of two on a 2D canvas before uploading.

4. Extend the shaders

Vertex shader. Declare a two-component UV attribute and a two-component varying, and assign one to the other. Nothing else in the transform changes.

Fragment shader. Declare the matching varying, a 2D sampler uniform and a boolean use-texture uniform. Sample the texture at the interpolated UV. Then combine:

Mode	Result	When to use
Replace	Output the texel color only; lighting is ignored.	Debugging UVs; the 'lighting off' toggle.
Modulate (recommended)	Multiply the ambient and diffuse terms by the texel color, and add the specular term unmodified.	Default. The texture reads as surface color, and highlights stay the color of the light.

Mode	Result	When to use
Modulate all	Multiply the full lit color by the texel.	Simpler, but highlights pick up the texture's color and look dull.

Integrate with the existing lighting toggle: when lighting is off, output the texel (textured sets) or the flat diffuse color (everything else), rather than the flat diffuse color alone. When the use-texture flag is off, the shader behaves exactly as today.

5. Hook up shader setup

After the program links, look up the UV attribute location and the sampler and flag uniform locations, and store them in shared state. Set the sampler uniform to texture unit 0 once; samplers are integers that name a unit, not the texture itself.

> **Note on attribute arrays.** The existing setup enables the position and normal attribute arrays once, for every draw. Do **not** do the same for the UV attribute: ellipsoids have no UV buffer, and an enabled attribute array with no buffer behind it makes draw calls fail. Enable and disable it per draw instead (step 6).

6. Bind per set in the renderer

Inside the triangle-set loop, after the material uniforms and before the draw call:

- Set the use-texture flag to true.
- Make texture unit 0 active and bind the set's texture to it.
- Enable the UV attribute array, bind the set's UV buffer and describe it as two floats per vertex, tightly packed.

Inside the ellipsoid loop, set the flag to false and disable the UV attribute array. With the array disabled, the shader reads a constant attribute value instead, which is harmless because the flag tells it to ignore the UV.

The ellipsoid problem

All objects share one shader program, so the UV attribute and sampler exist for ellipsoids too. There are three ways to handle them:

Option	Trade-off
Flag plus per-draw attribute enable (recommended)	Smallest change. One extra uniform, one enable or disable per draw. Ellipsoid rendering is unchanged.
Generate ellipsoid UVs	Lets ellipsoids be textured later. The mesh generator already walks longitude and latitude, so the longitude angle divided by a full turn gives U, and the latitude angle mapped from the south pole to the north pole gives V. Needs a duplicated seam column of vertices to avoid a smeared strip where U wraps from 1 back to 0.

Option	Trade-off
Two shader programs	Cleanest separation, but duplicates the setup and location lookup, and requires switching programs and re-sending shared uniforms each frame.

Pitfalls

Symptom	Cause and fix
Textures appear upside down	Image rows are stored top-down, but UV (0, 0) is the bottom left. Enable vertical flipping before uploading each image.
Sets render black	Usually an NPOT texture with mipmaps or repeat wrapping, or a sampler pointing at the wrong unit. Check the rules in step 3 and that unit 0 is active when binding.
Security error on upload	The image came from another origin without CORS permission, or the page was opened from the local file system, where local images count as cross-origin. Load images from the same CORS-enabled host as the scene data, or serve the project from a local web server.
Ellipsoids disappear or draws fail	The UV attribute array is still enabled while an ellipsoid is drawn. Disable it in the ellipsoid loop.
Sets flash white, then change	Expected: the placeholder shows until the image arrives. Use a neutral placeholder color, or wait for all images before starting the render loop.
Highlights look dull or tinted	The specular term is being multiplied by the texel. Add specular after modulation.
Seams or wrong mapping on a set	The UV list order does not match the vertex list order, or UVs were flattened with the wrong stride.

Testing checklist

- **Orientation.** A triangle set with an image that has readable text or a face shows it the right way up and not mirrored.
- **Mapping.** Temporarily output the UV as a color (U in red, V in green). Corners should go black, red, yellow and green, with smooth gradients between them.
- **Placeholder and failure.** Point one set at a missing image: it renders with the placeholder and logs one warning, and nothing else breaks.
- **NPOT.** At least one non-power-of-two image renders correctly.
- **Ellipsoids unchanged.** With texturing in place, ellipsoids look the same as before.
- **Toggles.** Lighting off shows plain texels on triangle sets; the triangle and ellipsoid toggles still hide and show their groups.

- **Interaction.** Highlighting, translating and rotating a textured set keeps the texture fixed to the surface, because UVs move with the vertices.
- **Console.** No WebGL warnings during the first frames or while images are loading.

Possible extensions

- **Transparency.** The materials already carry an alpha value, and PNG textures may have their own. Combining them needs blending, drawing opaque sets before transparent ones, and turning off depth writes for the transparent pass.
- **Combine-mode toggle.** A key or checkbox that switches between replace and modulate, useful for checking both lighting and mapping.
- **Textured ellipsoids.** Generate longitude and latitude UVs as described above and let ellipsoid materials name a texture too.